

Design Decisions & Architectural Choices

Overview

This document explains the key architectural decisions made while building the drone security system, the alternatives considered, and the reasoning behind each choice.

1. LangGraph for agent pipeline orchestration

Decision

Use LangGraph `StateGraph` with 6 typed nodes rather than a sequential chain of function calls.

Alternatives considered

- **Simple function pipeline:** `perceive(frame) -> contextualize() -> rule_check() -> alert()`
- **Celery/task queue:** Async tasks per processing stage
- **Monolithic processing function:** One large function handling all stages

Rationale

Conditional routing as first-class concept. The system needs to fork after rule evaluation: ambiguous rule hits go to LLM review, clear rule hits go straight to alert, and clean frames skip alerting entirely. In a sequential function pipeline this becomes nested conditionals scattered across unrelated code. LangGraph's `add_conditional_edges` expresses this as a single routing function with named outcomes.

Typed, inspectable state. `AgentState` TypedDict documents exactly what each node reads and writes. Bugs surface as `KeyError` or type mismatch at the boundary rather than `AttributeError: 'NoneType' deep inside a callback`.

Independent testability. Each node is a pure function `(state: dict) -> dict`. `test_agent_pipeline.py` stubs out the heavy models and exercises the full graph in milliseconds, while each node can be unit-tested by calling it directly with a hand-crafted state dict.

Future extensibility. Adding a CLIP anomaly detection branch, a human-in-the-loop review queue, or a parallel VLM + rule path would require adding one node and one edge — not refactoring the existing pipeline.

2. Two-tier VLM strategy (Bedrock primary / Moondream fallback)

Decision

Try Nova Lite via Amazon Bedrock first; fall back to Moondream2 local model transparently.

Alternatives considered

- **Single model (Bedrock only):** Simpler, but hard dependency on AWS credentials
- **Single model (local only):** Fully offline, but requires GPU for useful throughput
- **No VLM:** Caption-based rules (gate tampering, tool use) would be impossible

Rationale

Cost vs. quality. Nova Lite produces security-relevant 25-word captions in under 500ms at fractions of a cent per frame.

Local models capable of similar quality require a GPU and add latency.

Offline resilience. Security systems cannot tolerate a hard dependency on internet connectivity. Moondream2 (~2 GB) runs on CPU at ~1-2 fps — slow, but functional. The fallback activates automatically when Bedrock raises an exception.

Vendor flexibility. Switching the primary model (Nova Lite → GPT-4o Vision, for example) requires changing one environment variable (`VLM_CAPTIONER_MODEL`), not refactoring the captioner class.

3. Async VLM via ThreadPoolExecutor

Decision

Submit VLM caption jobs to a background thread pool; collect completed results on the next frame rather than waiting synchronously.

Alternative

Run VLM synchronously — simple, deterministic, but caps throughput at $1/\text{latency_vlm} \approx 2$ fps.

Rationale

YOLO + CLIP + rule evaluation takes ~50-80 ms per frame on a modern laptop CPU. VLM captioning via Bedrock takes ~500-1500 ms depending on network. Synchronous captioning would make the pipeline 10-20x slower.

The async design allows the hot path (YOLO + rules) to process at 12-20 fps while VLM results trickle in with ~500ms lag. Caption-based rules (gate tampering, tool use) accept this lag because the scenarios they detect require multiple seconds of evidence anyway — a one-frame delay is irrelevant.

Tradeoff: A caption-based rule may fire one frame later than the frame that actually triggered it. This is acceptable and documented.

4. Declarative YAML rule engine

Decision

Express all security rules in `configs/rules.yaml` with a structured condition schema, evaluated at runtime by `RuleEngine`.

Alternatives considered

- **Python rule functions:** `def after_hours_person(state) -> bool` — full language power but requires code changes for every new rule
- **Database-driven rules:** Store rules in SQLite — queryable but adds DB dependency to rule evaluation
- **ML classifier:** Train a binary classifier per threat type — high accuracy but requires labelled data and retraining cycle

Rationale

Operator autonomy. Security requirements change frequently. A `yaml` file edit + hot-reload (`engine.reload()`) deploys a new rule with zero code change, no pull request, no redeploy.

Auditability. The YAML is readable by non-developers. A security manager can verify that `after_hours_person` only fires between 22:00 and 06:00 without reading Python code.

LLM escalation flag. Any rule can set `needs_llm: true` to gate the alert on Nova's contextual judgment. This lets broad rules (e.g., `crowd_gathering`) avoid false positives without hand-tuning per-zone thresholds.

Hot-reload. `RuleEngine.reload()` atomically replaces the loaded rules without restarting the pipeline. This enables live rule tuning during an active incident.

5. Hybrid retrieval (ChromaDB semantic + SQLite structured)

Decision

Route natural language queries through intent parsing, then use ChromaDB (vector similarity) for semantic queries and SQLite (SQL filters) for temporal/class queries.

Alternative

Single CLIP-based retrieval for all queries.

Rationale

CLIP is excellent at answering *"show me suspicious activity near the gate"* (no explicit time/class) — the 512-dim embedding space captures visual semantics well. But CLIP fails at *"show me all trucks that entered on 13 April between 14:00 and 16:00"* — that requires exact timestamp and class matching that vector similarity cannot guarantee.

`SQL WHERE ts BETWEEN ? AND ? AND class = 'truck'` is $O(\log N)$ with indexes, 100% recall, and deterministic. No embedding inference needed at query time.

The intent parser (`_parse_query_intent`) uses a lightweight LLM call to extract structured fields (`has_time`, `start`, `end`, `class_filter`, `zone`). If `has_time=True`, the SQL path is used. Otherwise, ChromaDB is used. The cost of the intent parser call (~50ms / <0.01 cents) is justified by the dramatically better result quality for temporal queries.

6. Frame storage resolution separation

Decision

Model inputs use full-resolution source frames; saved JPEG thumbnails are compressed at a configurable preset (default 640×360 / quality 75).

Alternative

Save frames at model input resolution (e.g., 640×640 YOLO input).

Rationale

Detection quality must not be compromised. YOLOv8 and CLIP both perform best on their native input sizes. Downscaling the source image before inference would reduce detection confidence on small objects (people at distance, license plates).

Storage is a practical constraint. 1080p frames at 30fps = ~5 GB/hour uncompressed. The "Balanced" preset (640×360 / q75) reduces this to ~4 GB/hour → ~37 GB/day, which is more manageable for a security archive.

Thumbnails are for human review, not model inference. The saved JPEG is displayed in the Streamlit Timeline tab. 640×360 is sufficient for human visual confirmation of an alert. The full-resolution frame was already processed by the models before the thumbnail was saved.

7. Stream processing: file vs. RTSP modes

Decision

`drop_on_full=False` for file sources (reader blocks); `drop_on_full=True` for RTSP sources (reader evicts oldest).

Alternative

Single mode with a large queue (e.g., `queue_size=10000` for a 10-minute video).

Rationale

RTSP live streams must stay close to real-time. If the worker falls behind (e.g., during a slow Bedrock call), the system should skip old frames rather than accumulate a growing backlog that would make the system unresponsive.

Recorded files must process every frame. Dropping frames from a 5-minute clip defeats the purpose of reviewing the recording. The reader blocking until the worker has queue space ensures complete coverage at the cost of processing slower than real time.

Why not a large queue for files? The original implementation used `queue_size=max_frames` which would crash with `max_frames=None` (for unlimited processing) and would use excessive memory for long videos.

8. Alert deduplication cooldown

Decision

5-minute cooldown per `(rule_name, zone)` pair before re-firing the same alert.

Alternative

- No cooldown: every frame where the rule fires generates an alert
- Event-based deduplication: suppress while rule conditions still hold, fire again only after conditions clear

Rationale

Rules like `after_hours_person` would fire on every frame where a person is visible after 22:00 — potentially hundreds of alerts per minute. This would overwhelm any alert channel and make the system unusable.

5 minutes was chosen as a balance: it's long enough to prevent alert spam during a continuous event (a person loitering for 10 minutes generates 2 alerts, not 1800), and short enough that a genuinely new intrusion that starts shortly after a previous one is still reported.

False positives tagged with `[LLM: REVIEW]` are still stored (for later analysis) but also subject to the cooldown to prevent review-queue flooding.

9. In-memory SQLite connection caching for tests

Decision

When `db_path=":memory:"`, cache a single `sqlite3.Connection` in `self._mem_conn` and yield it from every `_get_conn()` call.

Why this was necessary

`sqlite3.connect(":memory:")` creates a fresh, empty database every time it is called. The original per-query connection strategy (open → execute → close → open → ...) applied the schema in connection #1 but then executed every subsequent query against connection #2, #3, etc. — each a fresh empty database with no tables.

This caused all integration and unit tests that used `SQLiteStore(":memory:")` to fail with `sqlite3.OperationalError: no such table: frames`.

The fix caches `self._mem_conn` for in-memory databases while keeping the per-call connection strategy for file-backed databases (where each connection sees the same on-disk state).

10. ByteTrack for cross-frame object identity

Decision

Use ByteTrack (built into Ultralytics) for persistent `track_id` assignment across frames.

Alternative

Re-detect objects independently per frame; use spatial proximity heuristics to match across frames.

Rationale

Loitering detection requires knowing that "person in frame 42" is the same physical person as "person in frame 120". Without a tracker, the system would treat every detection as a new object and duration-based rules would never fire.

ByteTrack uses Kalman filter prediction + IoU matching to maintain identity even through brief occlusions. It is included in the Ultralytics package so no additional dependency is required — `model.track()` replaces `model.predict()`.

11. Assumption: drone operates as a stationary overhead camera

Decision

The system treats the drone as fixed at a single GPS location for the duration of a patrol session. Zone names (`main_gate`, `perimeter_north`, etc.) are defined as static labels in the YAML configuration and in the `zone` field attached to every frame at ingest time.

Assumption

The drone is **stationary** — hovering at a fixed altitude and horizontal position. GPS coordinates produced by `TelemetrySimulator` vary only due to noise (± 0.3 m Gaussian jitter), not real displacement. Zone assignment is therefore stable: the zone label given to a frame at the start of the session remains valid for every subsequent frame from the same source.

What this means in practice

- Zone-based rules (`forbidden_zone`, `after_hours_zone_entry`, loitering thresholds) compare against a single constant zone string rather than computing which geographic polygon the drone is currently over.
- Frame retrieval filters (`zone="main_gate"`) produce consistent results because all frames from a single ingest run share the same zone label.
- The telemetry GPS values are stored and surfaced in the dashboard for operator awareness, but they do **not** drive zone assignment in the current implementation.

What would need to change for a truly mobile drone

If the drone is airborne and changes position during a patrol — covering multiple zones in a single flight — the following techniques would need to be implemented:

1. **Geofence polygons:** Define each security zone as a GPS polygon (GeoJSON). At every frame, compute which polygon (if any) the drone's current (`lat`, `lon`) falls inside using point-in-polygon intersection. Libraries such as `shapely` handle this efficiently.
2. **Dynamic zone tagging:** Replace the static `zone` parameter in `VideoIngestor / StreamProcessor` with a per-frame lookup: `zone = geofence.zone_for(telemetry["lat"], telemetry["lon"])`.

3. **Multi-zone events:** A single flight path crossing from `perimeter_north` to `main_gate` should generate separate event records per zone, not merge them under a single label.
4. **Zone-relative loitering:** The loitering timer in the `contextualize` node resets when the drone leaves and re-enters a zone, because `_track_first_seen` is keyed on `(zone, track_id)`. This behaviour is correct for a mobile drone but requires the dynamic zone tagging above to work properly.

Until a GPS-to-zone mapping layer is added, the system is designed and validated only for the fixed-position deployment model.

Technology Choice Summary

Decision	Chosen	Key Reason
Pipeline orchestration	LangGraph	Conditional routing + typed state + testable nodes
VLM primary	Bedrock Claude Haiku	Sub-second latency, no local GPU required
VLM fallback	Moondream2	Offline / no-credential environments
Object detection	YOLOv8s	Speed/accuracy balance; ByteTrack included
Semantic embeddings	OpenCLIP ViT-B/32	Pre-trained on broad visual concepts, zero-shot
Vector store	ChromaDB	Metadata filtering + HNSW speed + ephemeral mode for tests
Relational store	SQLite	Zero-config, WAL concurrent reads, portable
Rule format	YAML	Operator-editable, hot-reloadable, auditable
Dashboard	Streamlit	Rapid iteration, native dataframe/image rendering
Package manager	uv	Deterministic lockfile, fast installs, Python version management